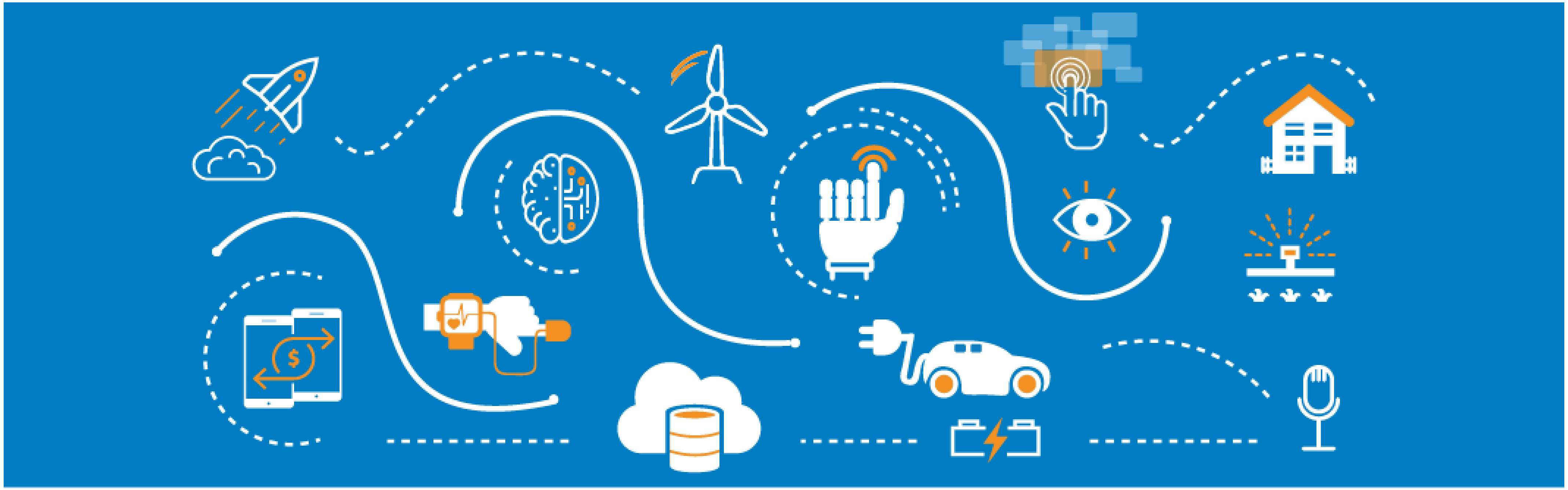


Computer Vision Part 4

Kien Tran

Contents

1. Dataset format and Dataset Preparation

1.1. Introduction

Pascal VOC and MS COCO are two most popular data format for object detection.

Most public available object detection datasets follow either one of these two formats.

- The Pascal Visual Object Classes (VOC) dataset for object detection and segmentation with 20 object classes and over 11K images.
- COCO: Common Objects in Context (COCO) is a large-scale object detection, segmentation, and captioning dataset with 80 object categories. (Over 200’000 images of the total 330’000 images are labeled and 1.5 Mio object instances)

Ref: https://docs.ultralytics.com/datasets/detect/#supported-datasets

1. Data Format For Object Detection

1.2. Popular data format

```
!tree -d

.
└── Pascal2012
    ├── VOC2012_test
    │   └── VOC2012
    │       ├── Annotations
    │       ├── ImageSets
    │       │   ├── Action
    │       │   ├── Layout
    │       │   ├── Main
    │       │   └── Segmentation
    │       └── JPEGImages
    └── VOC2012_train_val
        └── VOC2012
            ├── Annotations
            ├── ImageSets
            │   ├── Action
            │   ├── Layout
            │   ├── Main
            │   └── Segmentation
            └── JPEGImages
```

Dataset required - in Pascal VOC format

```
parent_dir
   |
   |---------train
   |           |
   |           |--------images
   |           |            |
   |           |            |--------img1.jpg
   |           |            |--------img2.jpg
   |           |            |---------....(And so on)
   |           |
   |           |---------annotations
   |                        |
   |                        |--------img1.xml
   |                        |--------img2.xml
   |                        |---------....(And so on)
   |
   |---------validation
               |
               |--------images
               |            |
               |            |--------img1.jpg
               |            |--------img2.jpg
               |            |---------....(And so on)
               |
               |---------annotations
                            |
                            |--------img1.xml
                            |--------img2.xml
                            |---------....(And so on)
```

```xml
<annotation>
        <folder>Kangaroo</folder>
        <filename>00001.jpg</filename>
        <path>./Kangaroo/stock-12.jpg</path>
        <source>
                <database>Kangaroo</database>
        </source>
        <size>
                <width>450</width>
                <height>319</height>
                <depth>3</depth>
        </size>
        <segmented>0</segmented>
        <object>
                <name>kangaroo</name>
                <pose>Unspecified</pose>
                <truncated>0</truncated>
                <difficult>0</difficult>
                <bndbox>
                        <xmin>233</xmin>
                        <ymin>89</ymin>
                        <xmax>386</xmax>
                        <ymax>262</ymax>
                </bndbox>
        </object>
</annotation>
```

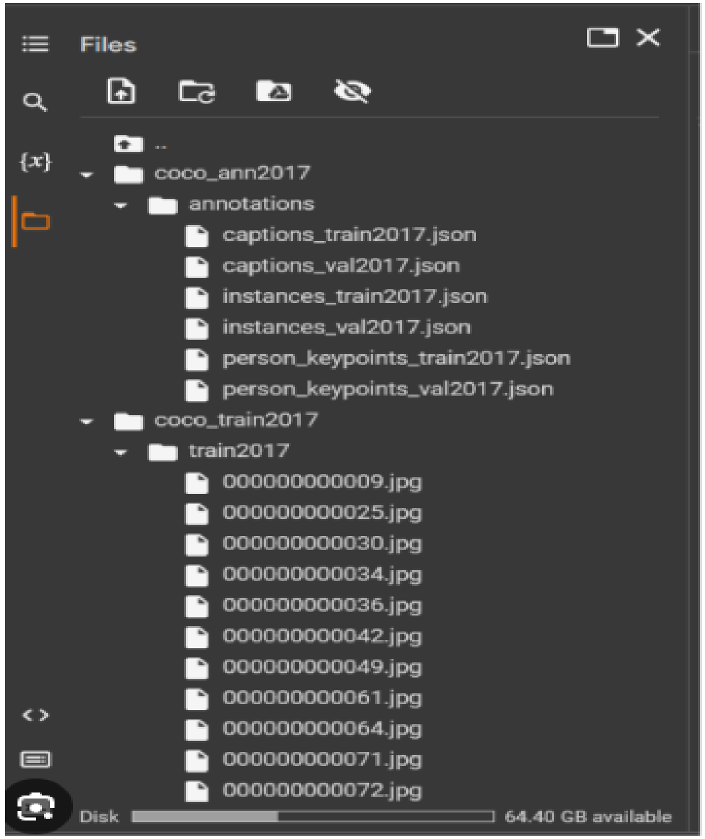
[VOC Format (XML file)]

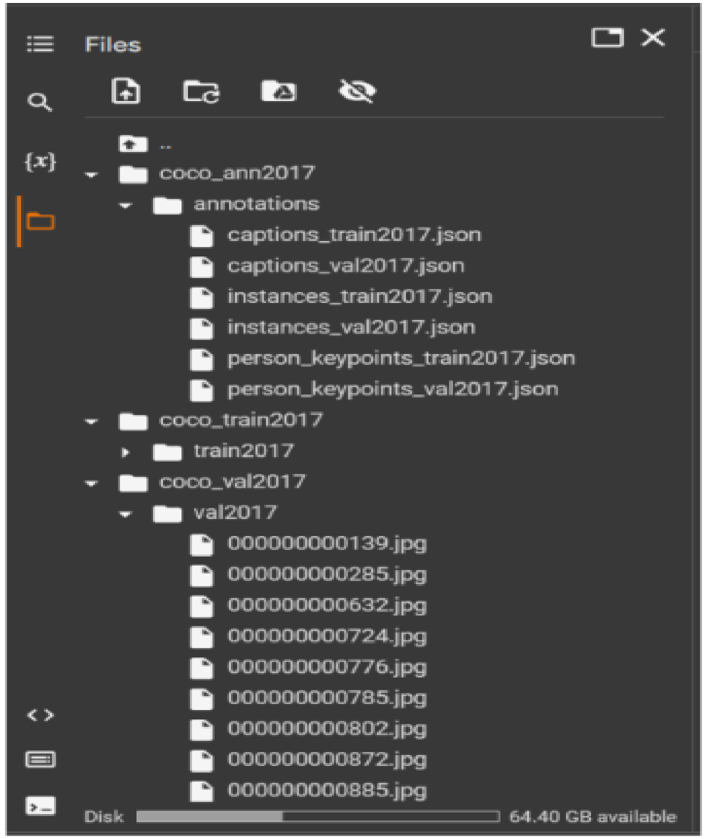
Sample Pascal VOC

Pascal VOC Bounding box :(*xmin-top left, ymin-top left,xmax-bottom right, ymax-bottom right*)

1. Data Format For Object Detection

1.2. Popular data format





COCO Format (JSON file)

COCO Bounding box: (*x-top left, y-top left, width, height*)

1. Data Format For Object Detection

1.2. Popular data format

```
{
    "info": info,
    "licenses": [license],
    "categories": [category],
    "images": [image],
    "annotations": [annotation]
}
```

Image: Contains list of all the images in dataset

```
image{
"id": int,
 "width": int,
 "height": int,
 "file_name": str,
 "license": int,
 "flickr_url": str,
 "coco_url": str,
 "date_captured": datetime,
}
```

```
"images": [
    {
        "id": 397133,
        "width": 640,
        "height": 640,
        "file_name": "101.jpg",
        "license":1,
         "date_captured": "2019-12-04 17:02:52"
    },
    {
         "id": 397122,
        "height": 640,
        "width": 640,
        "file_name": "102.jpg",
        "license": 1,
        "date_captured": "2019-12-04 17:02:52"
    }
    ]
```

template and example for images section of the json for COCO

Annoations: Contains list of each individual object annotation from every single image in the dataset

```
annotation{
"id": int,
 "image_id": int,
 "category_id": int,
 "segmentation": RLE or [polygon],
 "area": float,
 "bbox": [x,y,width,height],
 "iscrowd": 0 or 1,
}
```

```
"annotations": [
    {
        "segmentation":
[[510.66,423.01,511.72,420.03,...,510.45,423.01]],
        "area": 702.10,
        "iscrowd": 0,
        "image_id":  397133,
        "bbox": [433.07,355.93,138.65,228.67],
        "category_id": 18,
        "id": 1768
    },
    {
        "segmentation":
       {
         "counts":[12,56,198,10]
         "size":[120, 240]
       }
        "area": 500.2,
        "iscrowd": 1,
        "image_id": 397122,
        "bbox": [473.07,395.93,38.65,28.67],
        "category_id": 18,
        "id": 1768
    }]
```

template and example for annotations section of the JSON for COCO

Categories:

```
Categories
[{
"id": int,
 "name": str,
 "supercategory": str,
}]
```

```
"categories":
[
    {"id": 1,"name": "rose", "supercategory": "flower"},
    {"id": 2,"name": "tulip", "supercategory": "flower"},
    {"id": 10,"name": "Apple", "supercategory": "fruit"}
]
```

template and example for Categories section of the JSON for COCO

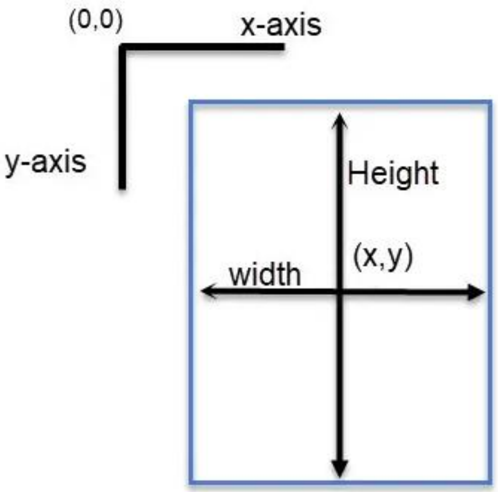
[COCO Format (JSON file)]

COCO Bounding box: (*x-top left, y-top left, width, height*)

1. Data Format For Object Detection

1.2. Popular data format

```
.
|__ datasets
|        |__ data.yaml
|        |__train
|        |        |__ images
|        |        |        |__ 1.jpg
|        |        |        |__ 2.jpg
|        |        |        |__ ..
|        |        |__ labels
|        |                 |__ 1.txt
|        |                 |__ 2.txt
|        |                 |__ ..
|        |__valid
|                 |__ images
|                 |        |__ 1.jpg
|                 |        |__ 2.jpg
|                 |        |__ ..
|                 |__ labels
|                          |__ 1.txt
|                          |__ 2.txt
|                          |__ ..
|__ train.py
|__ test.py
```



Bounding box format

```yaml
names:
- bus
- car
- motorcycle
- truck
nc: 4
roboflow:
  license: CC BY 4.0
  project: drug_bottle_detection
  url: https://universe.roboflow.com/tma-v5lto/hcm-data/dataset/1
  version: 1
  workspace: tma-v5lto
test: ../test/images
train: ../train/images
val: ../valid/images
```

File config: data.yml

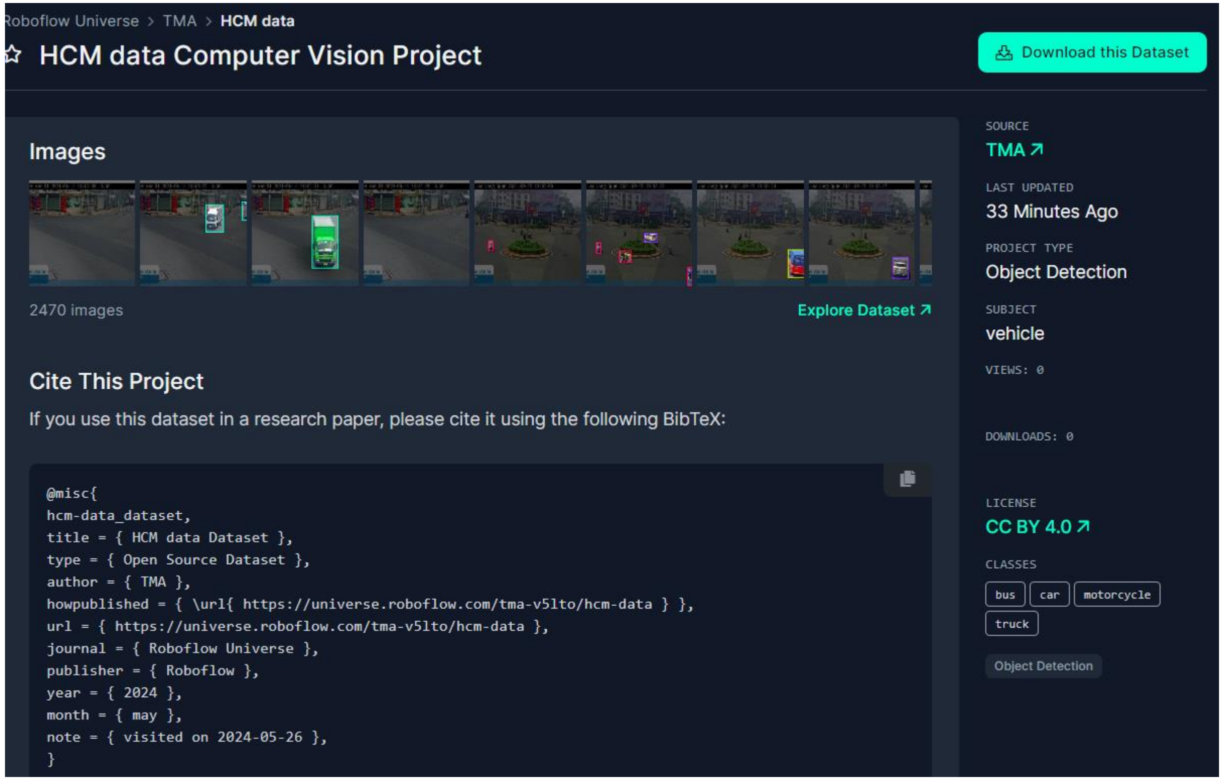
Sample folder stucture for training using Ultralytics package. (Source: Image by the author.)

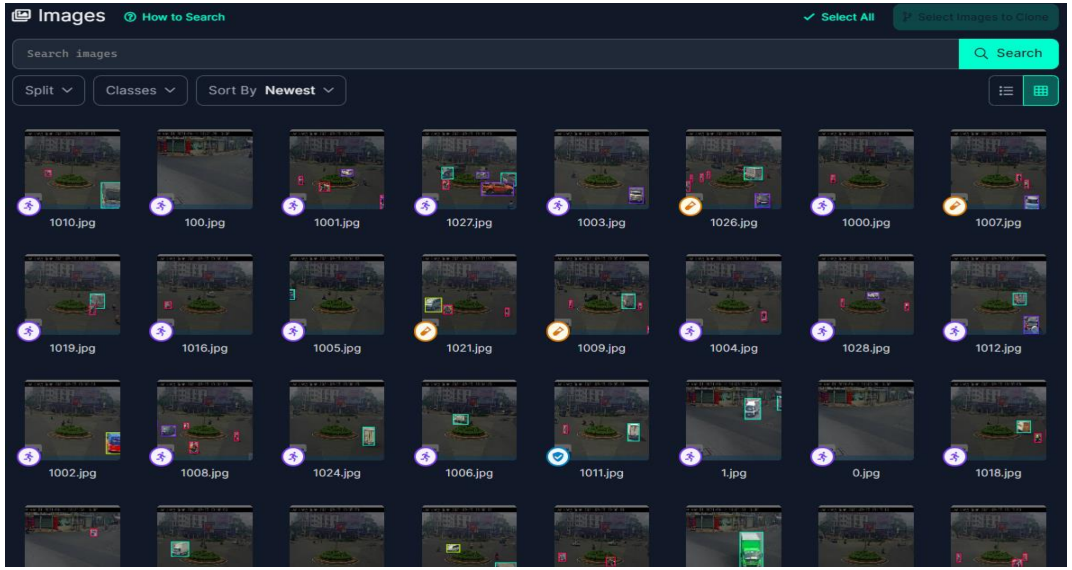
Yolo Format (txt file)

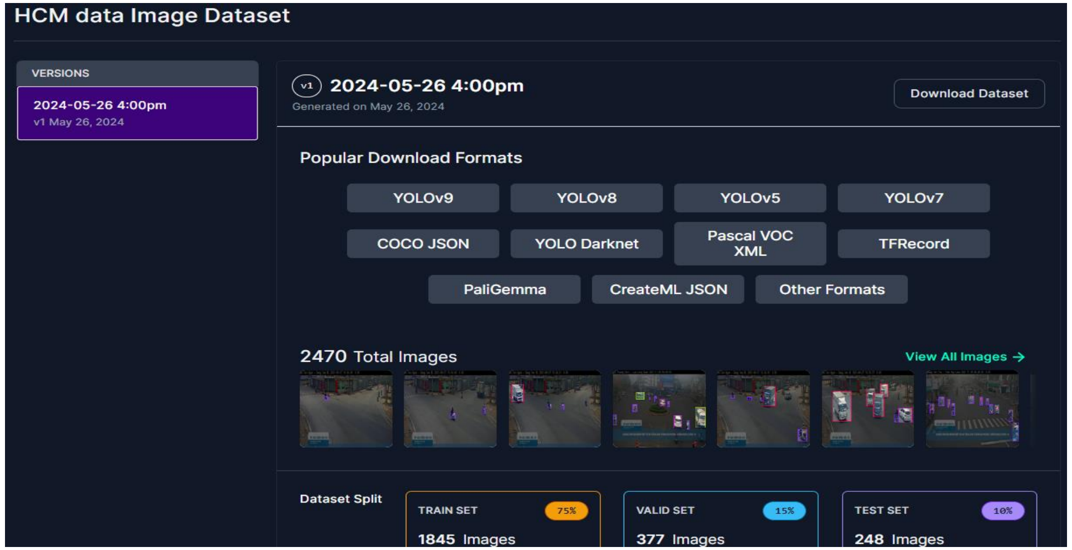
COCO Bounding box: (*x center, y center, width, height*)

1. Data Format For Object Detection

1.3. Prepare Data with Roboflow







https://roboflow.com

https://universe.roboflow.com/tma-v5lto/hcm-data/dataset/1

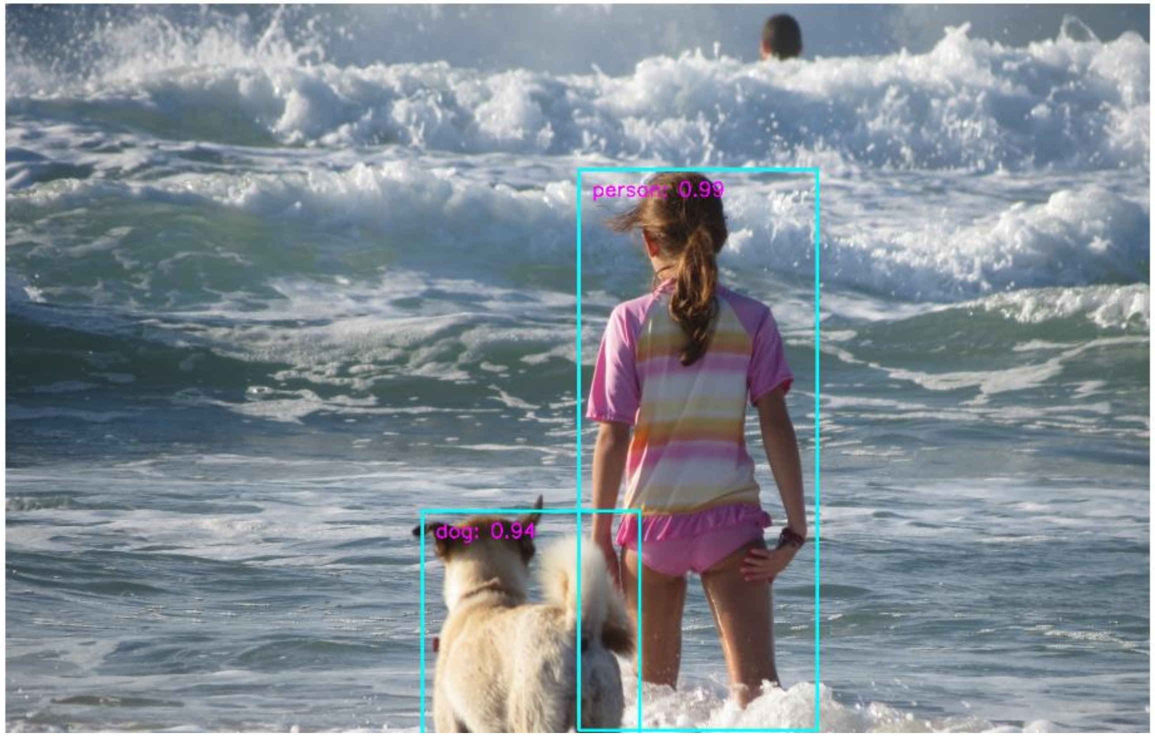
2. Training SSD Model with Custom Data

Single Shot MultiBox Detector Implementation in Pytorch

This repo implements SSD (Single Shot MultiBox Detector). The implementation is heavily influenced by the projects ssd.pytorch and Detectron. The design goal is modularity and extensibility.

Currently, it has MobileNetV1, MobileNetV2, and VGG based SSD/SSD-Lite implementations.

It also has out-of-box support for retraining on Google Open Images dataset.

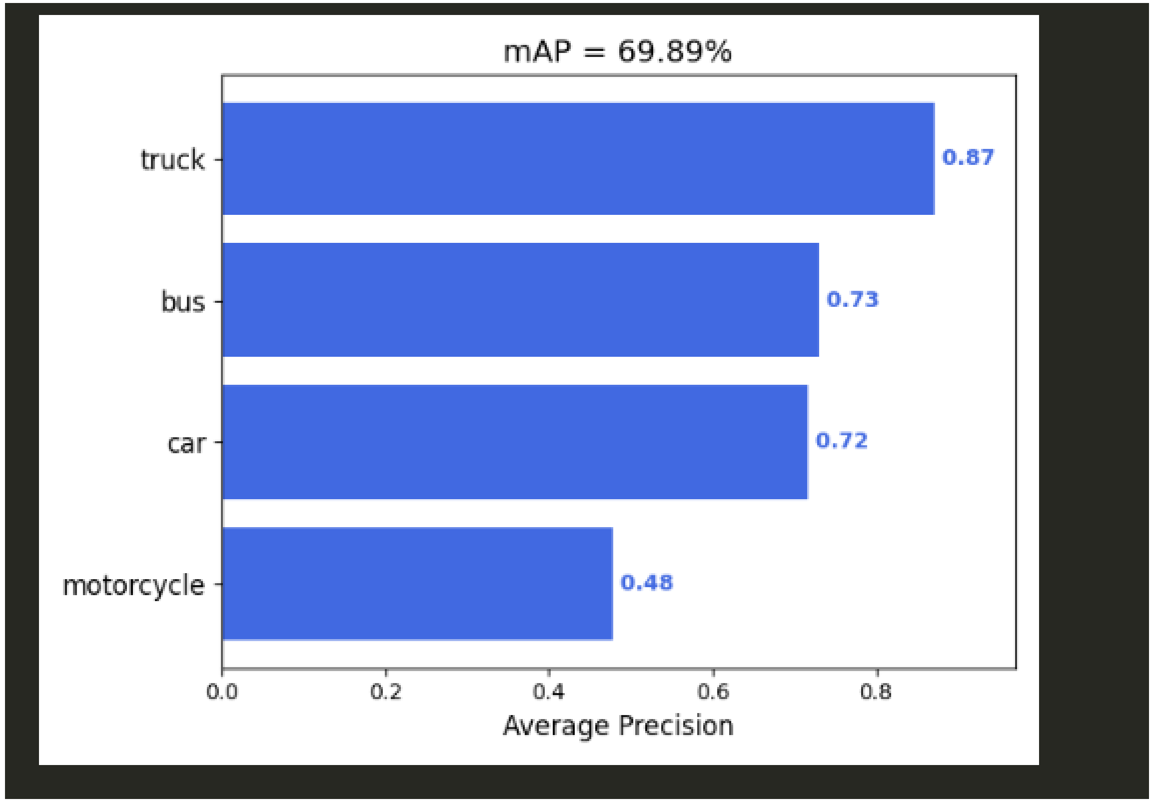


Ref: https://colab.research.google.com/drive/1acFO-dSF9Wo4m0YqgCeHma17NiwOIKuV?usp=sharing

Ref: https://github.com/qfgaohao/pytorch-ssd/tree/master

2. Training SSD Model with Custom Data

```
2021-12-07 02:40:30,608 - root - INFO - Epoch: 0, Validation Loss: 3.6739, Validation Regression Loss 1.1796, Validation Classification Loss: 2.4943
2021-12-07 02:40:30,799 - root - INFO - Saved model models/vgg16-ssd-Epoch-0-Loss-3.6738986587524414.pth
2021-12-07 02:56:42,228 - root - INFO - Epoch: 5, Validation Loss: 2.6860, Validation Regression Loss 0.7542, Validation Classification Loss: 1.9318
2021-12-07 02:56:42,394 - root - INFO - Saved model models/vgg16-ssd-Epoch-5-Loss-2.6860147094726563.pth
2021-12-07 03:12:50,147 - root - INFO - Epoch: 10, Validation Loss: 2.4596, Validation Regression Loss 0.6233, Validation Classification Loss: 1.8363
2021-12-07 03:12:50,312 - root - INFO - Saved model models/vgg16-ssd-Epoch-10-Loss-2.459552125930786.pth
2021-12-07 03:28:56,121 - root - INFO - Epoch: 15, Validation Loss: 2.3499, Validation Regression Loss 0.5893, Validation Classification Loss: 1.7606
```

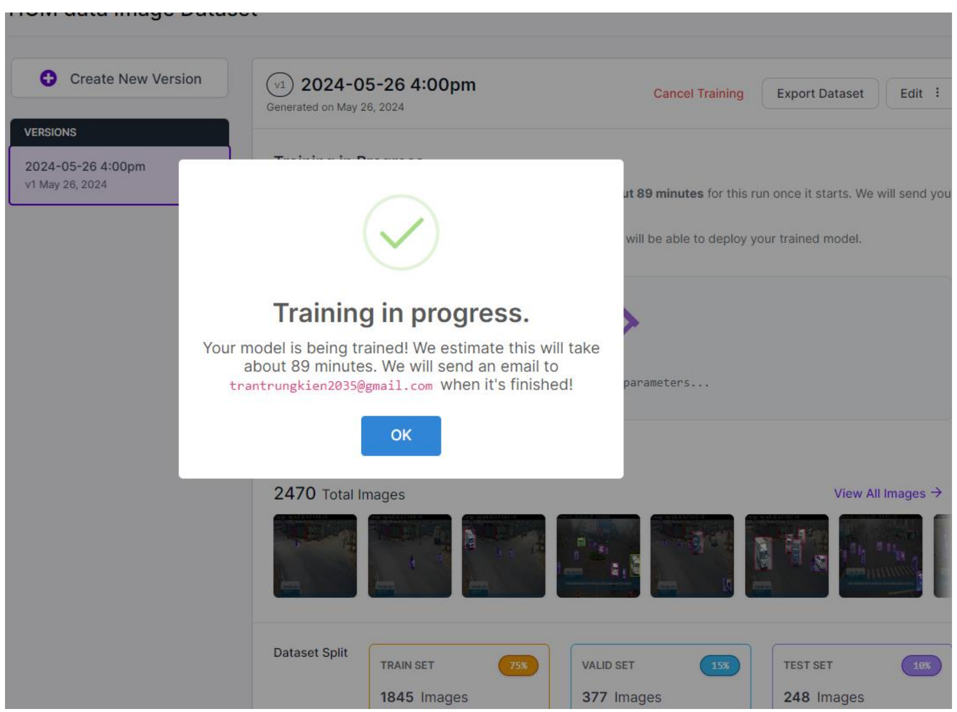


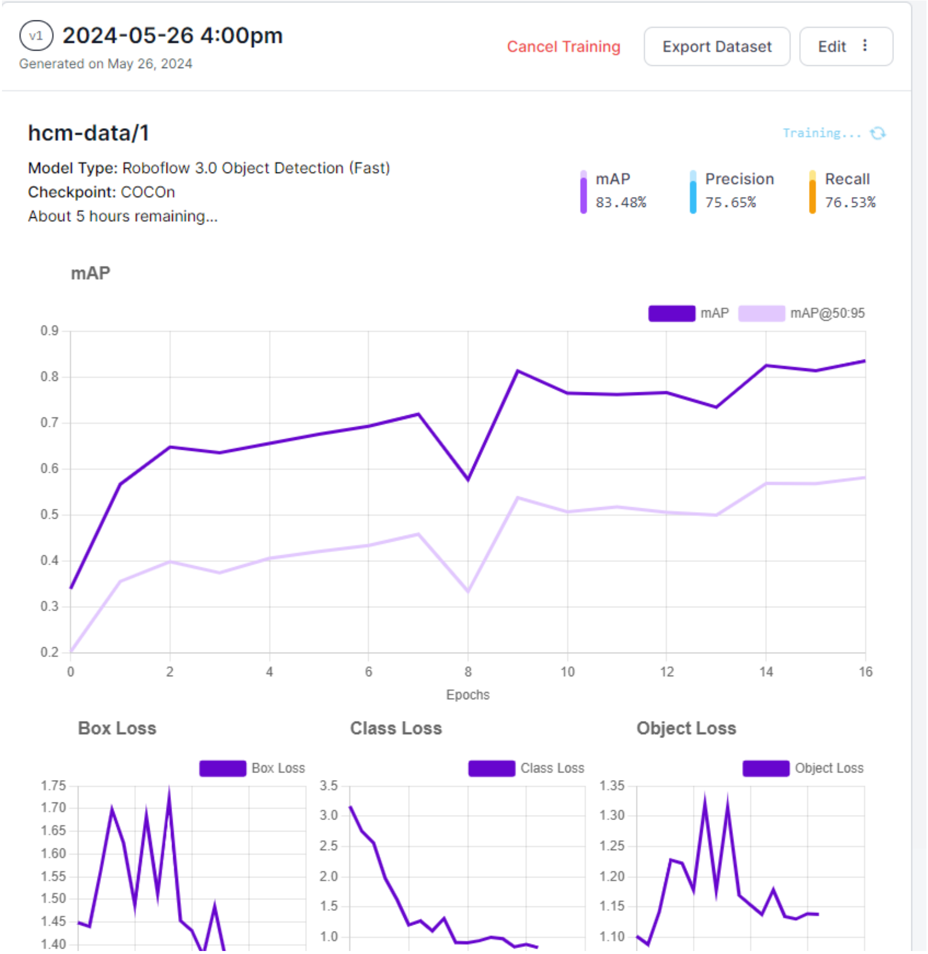
Ref: https://colab.research.google.com/drive/1acFO-dSF9Wo4m0YqgCeHma17NiwOIKuV?usp=sharing

Ref: https://github.com/qfgaohao/pytorch-ssd/tree/master

3. Training Yolov7 Model with Custom Data

Training with Roboflow

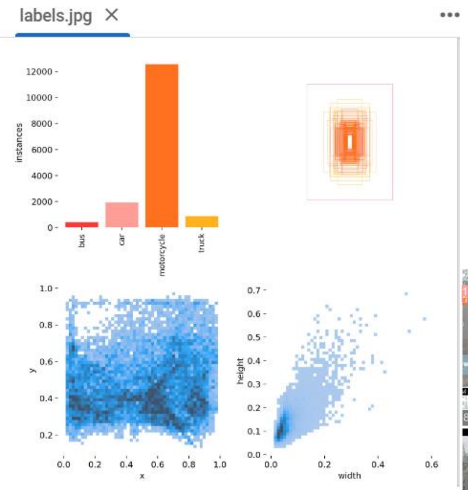




Ref: https://colab.research.google.com/drive/1acFO-dSF9Wo4m0YqgCeHma17NiwOIKuV?usp=sharing

4. Training Yolov8 Model with Custom Data

```
Downloading https://ultralytics.com/assets/Arial.ttf to '/root/.config/Ultralytics/Arial.ttf'...
100% 755k/755k [00:00<00:00, 12.8MB/s]
Overriding model.yaml nc=80 with nc=4

                   from  n    params  module                                       arguments
  0                  -1  1      1392  ultralytics.nn.modules.conv.Conv             [3, 48, 3, 2]
  1                  -1  1     41664  ultralytics.nn.modules.conv.Conv             [48, 96, 3, 2]
  2                  -1  2    111360  ultralytics.nn.modules.block.C2f             [96, 96, 2, True]
  3                  -1  1    166272  ultralytics.nn.modules.conv.Conv             [96, 192, 3, 2]
  4                  -1  4    813312  ultralytics.nn.modules.block.C2f             [192, 192, 4, True]
  5                  -1  1    664320  ultralytics.nn.modules.conv.Conv             [192, 384, 3, 2]
  6                  -1  4   3248640  ultralytics.nn.modules.block.C2f             [384, 384, 4, True]
  7                  -1  1   1991808  ultralytics.nn.modules.conv.Conv             [384, 576, 3, 2]
  8                  -1  2   3985920  ultralytics.nn.modules.block.C2f             [576, 576, 2, True]
  9                  -1  1    831168  ultralytics.nn.modules.block.SPPF            [576, 576, 5]
 10                  -1  1         0  torch.nn.modules.upsampling.Upsample         [None, 2, 'nearest']
 11             [-1, 6]  1         0  ultralytics.nn.modules.conv.Concat           [1]
 12                  -1  2   1993728  ultralytics.nn.modules.block.C2f             [960, 384, 2]
 13                  -1  1         0  torch.nn.modules.upsampling.Upsample         [None, 2, 'nearest']
 14             [-1, 4]  1         0  ultralytics.nn.modules.conv.Concat           [1]
 15                  -1  2    517632  ultralytics.nn.modules.block.C2f             [576, 192, 2]
 16                  -1  1    332160  ultralytics.nn.modules.conv.Conv             [192, 192, 3, 2]
 17            [-1, 12]  1         0  ultralytics.nn.modules.conv.Concat           [1]
 18                  -1  2   1846272  ultralytics.nn.modules.block.C2f             [576, 384, 2]
 19                  -1  1   1327872  ultralytics.nn.modules.conv.Conv             [384, 384, 3, 2]
 20             [-1, 9]  1         0  ultralytics.nn.modules.conv.Concat           [1]
 21                  -1  2   4207104  ultralytics.nn.modules.block.C2f             [960, 576, 2]
 22        [15, 18, 21]  1   3778012  ultralytics.nn.modules.head.Detect           [4, [192, 384, 576]]
Model summary: 295 layers, 25858636 parameters, 25858620 gradients, 79.1 GFLOPs
```



Ref: https://colab.research.google.com/drive/1acFO-dSF9Wo4m0YqgCeHma17NiwOIKuV?usp=sharing

Ref: https://github.com/qfgaohao/pytorch-ssd/tree/master

4. Training Yolov8 Model with Custom Data

```
                  all        377       3021      0.815       0.85      0.878      0.639
EarlyStopping: Training stopped early as no improvement observed in last 30 epochs. Best results observec
To update EarlyStopping(patience=30) pass a new patience value, i.e. `patience=300` or use `patience=0` t
```

Notebook

```
101 epochs completed in 2.072 hours.
Optimizer stripped from runs/detect/train/weights/last.pt, 52.0MB
Optimizer stripped from runs/detect/train/weights/best.pt, 52.0MB

Validating runs/detect/train/weights/best.pt...
Ultralytics YOLOv8.2.22 🚀 Python-3.10.12 torch-2.3.0+cu121 CUDA:0 (Tesla T4, 15102MiB)
Model summary (fused): 218 layers, 25842076 parameters, 0 gradients, 78.7 GFLOPs
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% 6/6 [00:1
                   all        377       3021      0.829      0.843      0.887      0.648
                   bus        377         81      0.834      0.827      0.828      0.663
                   car        377        391      0.828      0.859      0.914      0.686
            motorcycle        377       2372      0.817      0.829      0.896      0.538
                 truck        377        177      0.835      0.859      0.911      0.704
Speed: 0.3ms preprocess, 11.1ms inference, 0.0ms loss, 6.3ms postprocess per image
Results saved to runs/detect/train
💡 Learn more at https://docs.ultralytics.com/modes/train
```

Ref: https://colab.research.google.com/drive/1acFO-dSF9Wo4m0YqgCeHma17NiwOIKuV?usp=sharing

Ref: https://github.com/qfgaohao/pytorch-ssd/tree/master

5. Training Yolov9 v10 Model with Custom Data

Yolov9

Trainning

```
!yolo task=detect \
mode=train \
model=/content/drive/MyDrive/Colab/training_hcm/yolov9c.pt \
data=/content/drive/MyDrive/Colab/training_hcm/hcm_data_yolov8/yolov7_9_data.yaml \
patience=30 \
batch=32 \
epochs=150 \
imgsz=640 \
save=True \
resume=True \
project=/content/drive/MyDrive/Colab/training_hcm/hcm_data_yolov8/result_train9 \
plots=True \
device=0
```

```
Ultralytics YOLOv8.2.22 🚀 Python-3.10.12 torch-2.3.0+cu121 CUDA:0 (Tesla T4, 15102MiB)
```

Ref: https://drive.google.com/file/d/1dHkc3iM-XYXhhzB9wgLihUhr6n42OTmc/view?usp=sharing

Yolov10

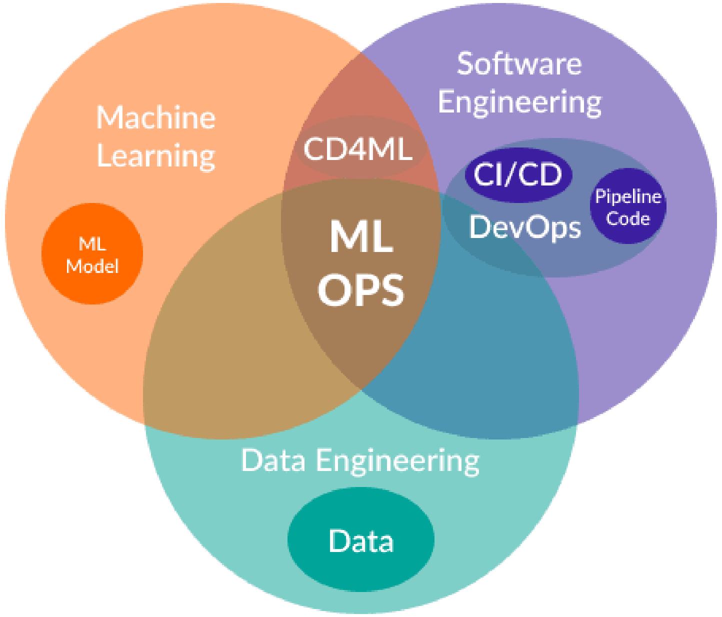
Trainning

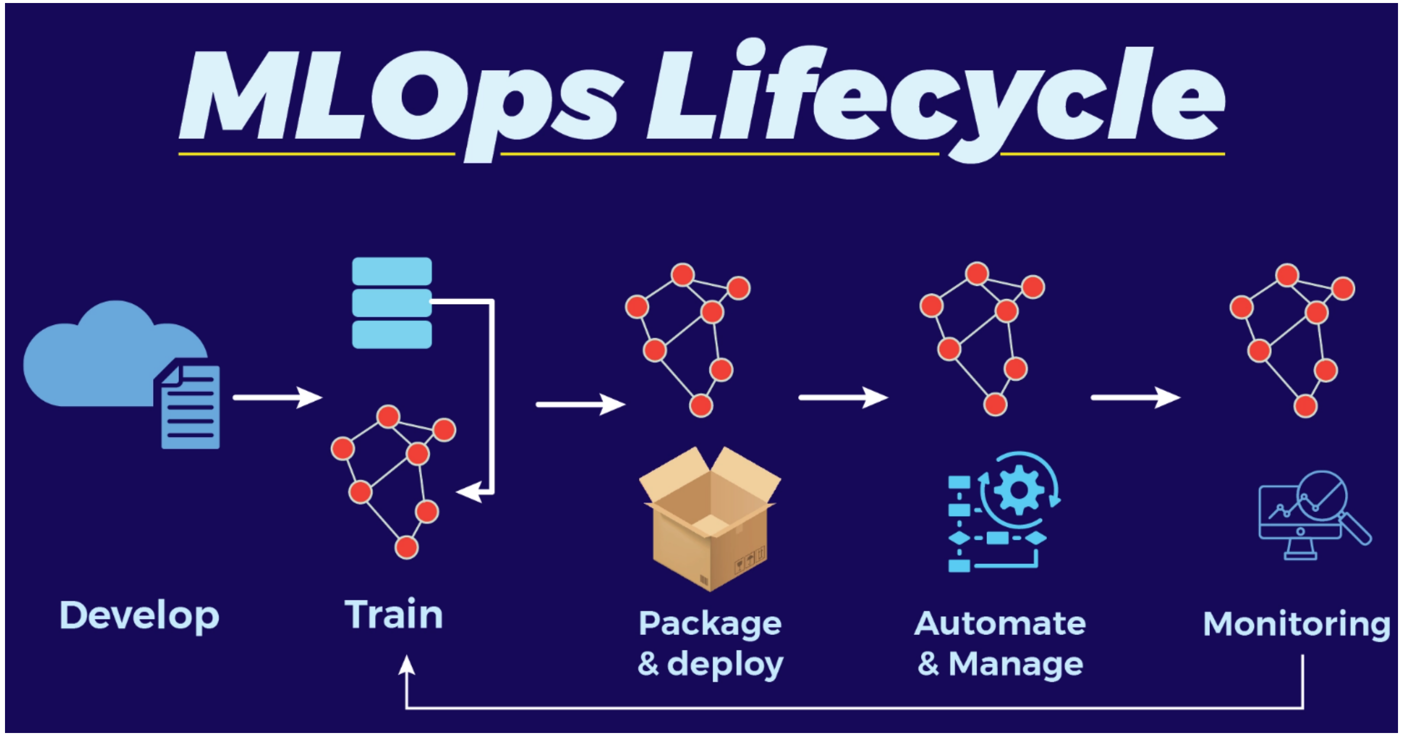
```
!yolo task=detect \
mode=train \
model=/content/drive/MyDrive/Colab/training_hcm/yolov10m.pt \
data=/content/drive/MyDrive/Colab/training_hcm/hcm_data_yolov8/yolov7_9_data.yaml \
patience=30 \
batch=32 \
epochs=150 \
imgsz=640 \
save=True \
resume=True \
project=/content/drive/MyDrive/Colab/training_hcm/hcm_data_yolov8/result_train10 \
plots=True \
device=0
```

```
Ultralytics YOLOv8.2.22 🚀 Python-3.10.12 torch-2.3.0+cu121 CUDA:0 (Tesla T4, 15102MiB)
```

Ref: https://drive.google.com/file/d/1b1Qk3kqOizXZ5tnNMao8BwSU32hUpBA9/view?usp=sharing

6.0. MLops





MLOps stands for Machine Learning Operations. MLOps is a core function of Machine Learning engineering, focused on streamlining the process of taking machine learning models to production, and then maintaining and monitoring them. MLOps is a collaborative function, often comprising data scientists, devops engineers, and IT.

Ref: https://clear.ml/docs/latest/

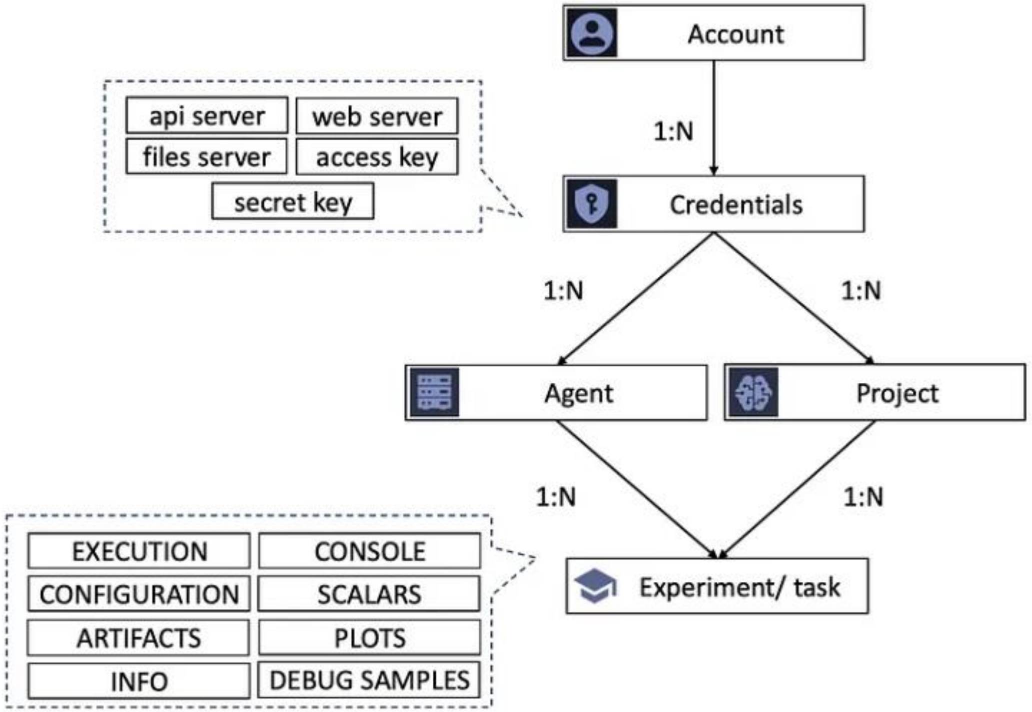
6. Project Lab: MLops

6.1. Basic concept

ClearML is an open source platform that automates and simplifies developing and managing machine learning solutions.

Key features:

- Real-time metrics tracking (loss, accuracy, validation)
- Experiment comparison
- Detailed logs and outputs
- Resource utilization monitoring
- Model Artifacts Management



Structure of ClearML

Ref: https://clear.ml/docs/latest/

6.2. Set up

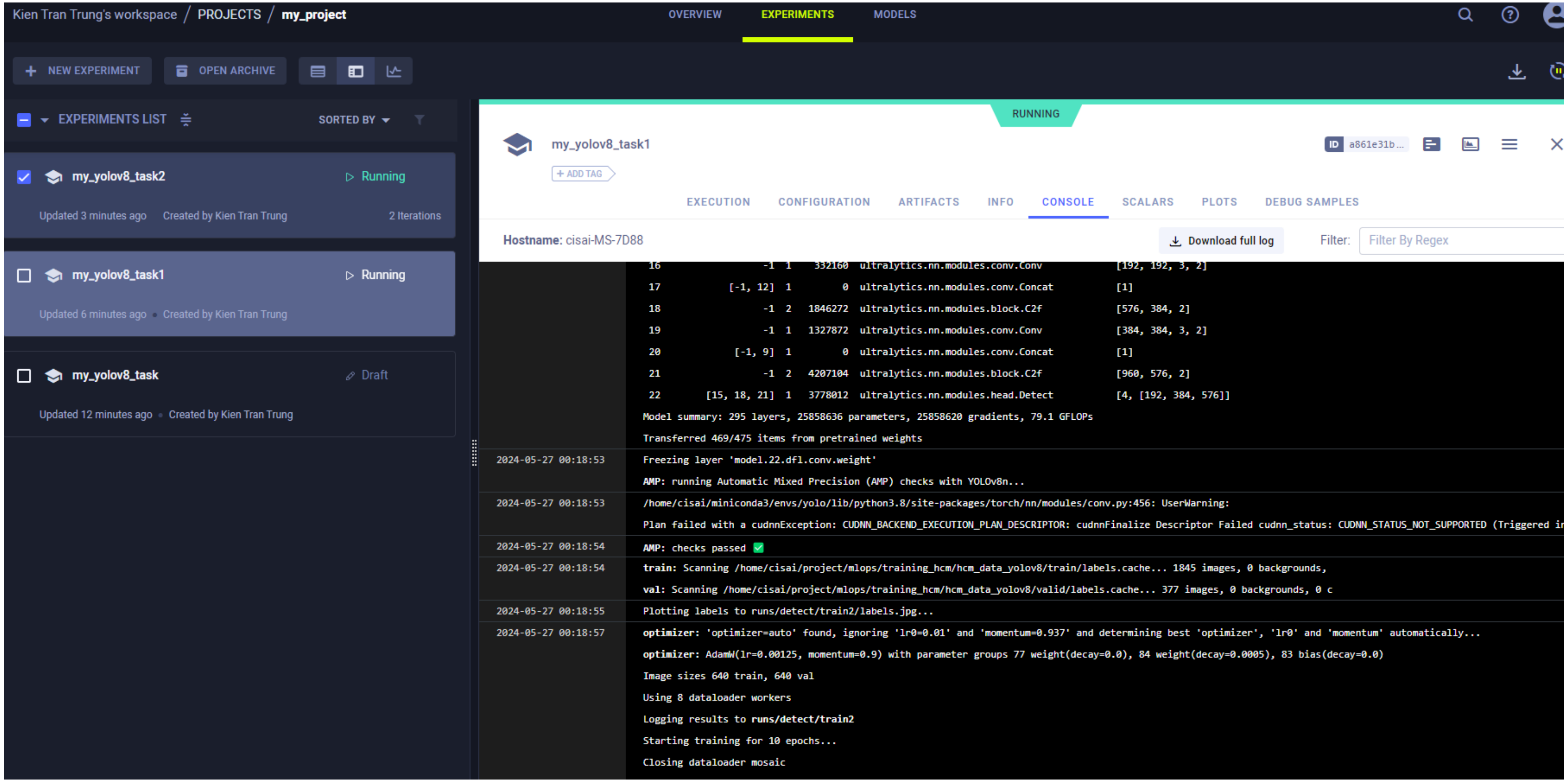
```
!pip install clearml
from clearml import Task, Logger
```

```
# Method 1) Set the environmental variables

%env CLEARML_WEB_HOST=https://app.clear.ml
%env CLEARML_API_HOST=https://api.clear.ml
%env CLEARML_FILES_HOST=https://files.clear.ml
%env CLEARML_API_ACCESS_KEY= <your access key>
%env CLEARML_API_SECRET_KEY= <your secret key>

# Method 2) Specify in Task object

Task.set_credentials(
     api_host="https://api.clear.ml",
     web_host="https://app.clear.ml",
     files_host="https://files.clear.ml",
     key='<your access key>',
     secret='<your secret key>'
)
```

Ref: https://docs.ultralytics.com/integrations/clearml/#configuring-clearml

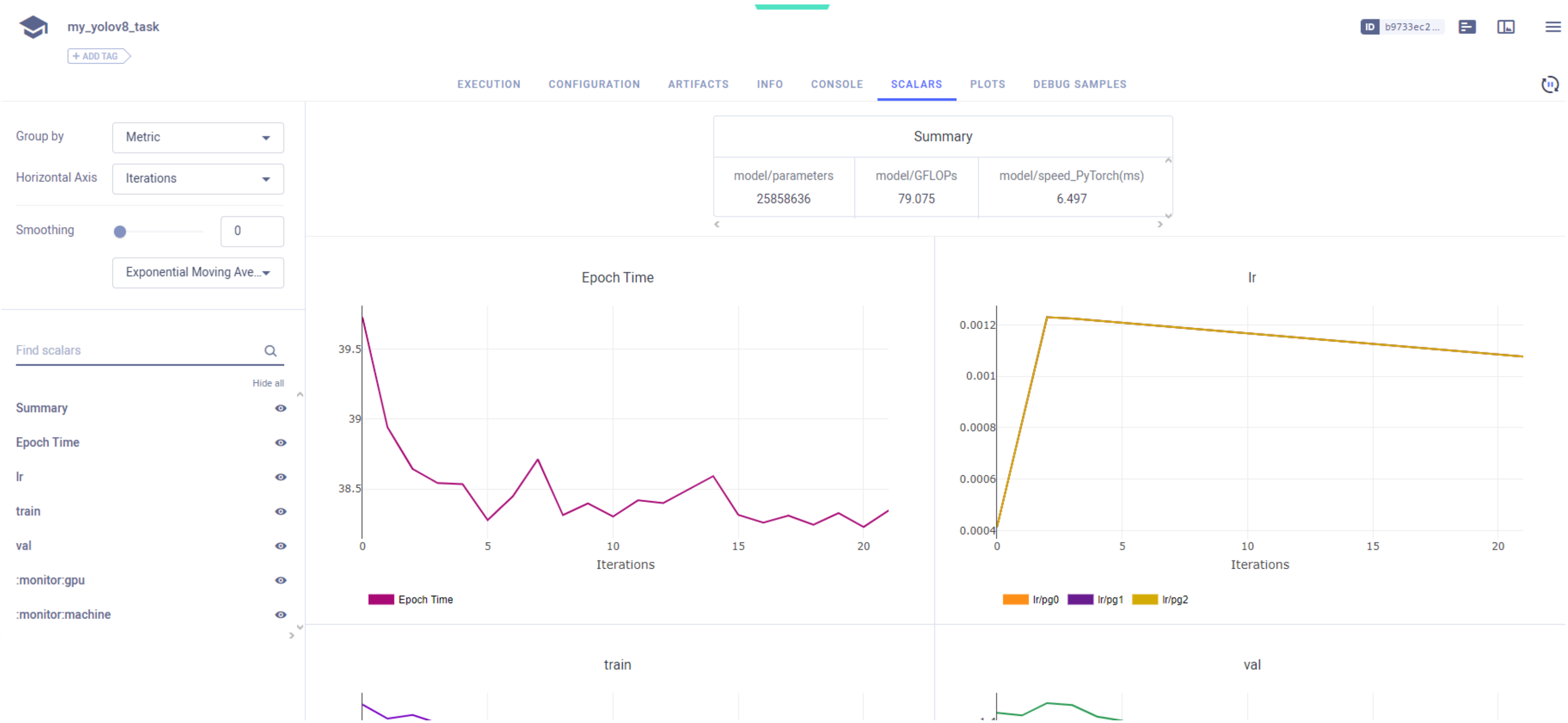
6. Project Lab: MLops

6.3. Monitoring



Ref: https://docs.ultralytics.com/integrations/clearml/#configuring-clearml

6.3. Monitoring



Ref: https://docs.ultralytics.com/integrations/clearml/#configuring-clearml

THANK YOU!

Email: contact@tmainnovation.vn

Website: www.tmainnovation.vn

TMA innovation